

QA CASE STUDY

Authentication & Progressive Lockout Testing

Enterprise HR & Payroll System

An anonymized QA case study demonstrating functional, negative, boundary, regression, and security-focused testing.

Role: QA Analyst

Testing: Manual, Functional, Regression, Security

Tools: Azure DevOps, Swagger, Postman, Cursor AI

CONFIDENTIAL PROJECT | ANONYMIZED FOR PORTFOLIO

Confidentiality: This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Project Overview

The HR & Payroll system included an existing account lockout mechanism designed to protect user accounts from repeated invalid login attempts.

The team introduced an enhancement that changed the existing fixed lockout behavior to a progressive lockout model, where the lockout duration increases as unsuccessful login attempts continue.

QA Objective

Validate that the new progressive lockout behavior correctly protects user accounts while maintaining accurate user messaging, consistent authentication states, and existing login functionality.

- Security — ensure repeated failed attempts trigger the correct protection.
- Accuracy — ensure lockout duration and user messaging match the configured rules.
- Stability — ensure the enhancement does not negatively affect existing authentication workflows.

My QA Contributions

- Test Design — created test suites and test cases
- Functional Testing — validated login and lockout behaviour
- Negative Testing — tested invalid credentials and edge cases
- Defect Management — identified, documented, and retested defects
- Regression Testing — validated related authentication workflows
- Collaboration — worked with Dev, Scrum Master, Release Manager, and TPM

The Feature Change

Before: Fixed Lockout	After: Progressive Lockout
Invalid attempts → repeated failures → fixed threshold → account lockout	1st–3rd attempts: no lockout
	4th attempt: 60 seconds
	5th attempt: 5 minutes
	6th attempt: 10 minutes
	7th attempt: 15 minutes
	8th attempt: 30 minutes
	9th attempt: 4 hours
	10th attempt: indefinite

QA Focus: Each transition point represented a different authentication state and required independent validation.

Expected Lockout Matrix

Failed Attempts	Expected Behavior
1st – 3rd	No lockout
4th	60 seconds lockout
5th	5 minutes lockout
6th	10 minutes lockout
7th	15 minutes lockout
8th	30 minutes lockout
9th	4 hours lockout
10th	Indefinite lockout

Confidentiality: This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Test Scope & Environment

In Scope

- Login authentication
- Failed-attempt tracking
- Progressive lockout
- Lockout duration
- Lockout messaging
- Session timeout
- Account recovery
- Related authentication workflows

Out of Scope

- Payroll calculations
- Employee records
- Other HR modules
- Unrelated application functionality

Test Environment

Environment: UAT

Application: Enterprise HR & Payroll System

Testing Type: Manual

API Validation: Postman / Swagger

Defect Tracking: Azure DevOps

Entry Criteria

- Feature requirements were available
- Build was deployed to UAT
- Test environment was accessible
- Test cases were prepared

Exit Criteria

- Critical scenarios passed
- Reported defects were resolved and retested
- Regression testing was completed
- No blocking issues remained for release validation

***Confidentiality:** This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.*

QA Risk Areas

The feature is authentication-related, so certain tests were prioritized based on risk.

Risk	QA Response
Incorrect lockout threshold	Boundary testing
Incorrect duration	Progressive lockout validation
Incorrect user message	Expected vs. actual comparison
Session state inconsistency	Session timeout scenarios
Regression in login	Authentication regression suite

QA Challenge

The system needed to keep multiple states consistent: failed attempt count, lockout rule, actual account state, displayed message, and session state.

Key QA Questions

- Does the correct attempt count trigger the correct lockout?
- Does the actual lockout duration match the configured behaviour?
- Does the user see the correct lockout message?
- What happens when the session expires?
- Does fixing one lockout scenario affect another?

Test Strategy

A combination of test types was used to ensure quality.

- Functional — validated login behavior, failed attempts, lockout rules, and recovery.
- Negative — tested invalid credentials, repeated failures, locked accounts, and login attempts during lockout.
- Boundary — focused on transition points such as the 3rd → 4th and 9th → 10th attempts.
- Regression — validated related authentication workflows after defects were fixed.
- Security — verified account protection rules and authentication state consistency.

Confidentiality: This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

Progressive Lockout Test Scenarios

ID	Scenario	Expected Result
TC-001	Valid login	User logs in successfully
TC-002	1 invalid attempt	Login rejected
TC-003	3 invalid attempts	No lockout
TC-004	4th invalid attempt	60-second lockout
TC-005	5th invalid attempt	5-minute lockout
TC-006	6th invalid attempt	10-minute lockout
TC-007	7th invalid attempt	15-minute lockout
TC-008	8th invalid attempt	30-minute lockout
TC-009	9th invalid attempt	4-hour lockout
TC-010	10th invalid attempt	Indefinite lockout
TC-011	Login during lockout	Login remains blocked
TC-012	Session timeout	Correct state and message

Boundary testing was prioritized because each threshold changed the expected behaviour.

Defects Identified

During execution, key issues were discovered.

Defect 01 — Lockout Message Did Not Match Failed Attempt Count

Severity: High **Priority:** High

The displayed lockout message did not always correspond to the current invalid-login attempt and expected lockout duration.

Expected: “Account locked for 60 seconds.”

Actual: “Invalid credentials.”

Impact: Users could receive incorrect information about their account status and recovery time.

QA Actions: Validated attempt count, lockout duration, and displayed message together.

***Confidentiality:** This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.*

Defect 02 — Incorrect Lockout Message After Session Timeout

Severity: High Priority: High

When the user session timed out, the system could display an incorrect lockout-related message that did not accurately represent the current authentication state.

Impact: Users could receive misleading information after session expiration.

QA Actions: Validated account state, attempt count, session timeout, and displayed message.

Sample Bug Report (Azure DevOps Ticket)

Field	Detail
ID	BUG-001
Title	Lockout Message Did Not Match Failed Attempt Count
Severity / Priority	High / High
Environment	UAT
Initial Test	Failed
Retest	Passed
Status	Resolved

Reproduction steps: open the login page, enter invalid credentials, repeat until the 4th failed attempt, continue through the progressive lockout thresholds, and compare the displayed message with the expected lockout duration.

Expected Result: Lockout duration and message should match the current failed-attempt count.

Actual Result: Lockout message displayed an incorrect duration for the current attempt.

Acceptance criteria: given a user reaches a configured progressive lockout threshold, when the account becomes locked due to consecutive failed login attempts, then the system should apply the corresponding lockout duration based on the attempt count, display a message that accurately reflects the applied lockout duration, and prevent the user from logging in during the lockout period.

Defect Lifecycle

From detection to release validation: Test → Defect Identified → Azure DevOps Log → Developer Investigation → Fix Implemented → Retest → Regression → Release Validation.

A defect was not considered complete after the initial fix. The affected scenario was retested, followed by regression testing of related authentication functionality.

Sample Retest Task

Objective: verify that the reported lockout message defect was fixed and the correct message is displayed for each progressive lockout level.

ID	Test	Expected Result	Result
RT-001	Trigger 4th failed attempt	60-sec lockout message displayed	Pass
RT-002	Trigger 5th failed attempt	5-min lockout message displayed	Pass
RT-003	Trigger 6th failed attempt	10-min lockout message displayed	Pass
RT-004	Trigger 7th failed attempt	15-min lockout message displayed	Pass
RT-005	Trigger 8th failed attempt	30-min lockout message displayed	Pass
RT-006	Trigger 9th failed attempt	4-hour lockout message displayed	Pass
RT-007	Trigger 10th failed attempt	Indefinite lockout message displayed	Pass

Retest result: PASS — the lockout message correctly corresponded to the failed-attempt count and configured lockout duration after the fix.

Sample Regression

Objective: verify that the lockout fix and implementation changes did not negatively affect existing authentication functionality.

ID	Regression Test	Expected Result	Result
REG-001	Login with valid credentials	User logs in successfully	Pass
REG-002	Login with invalid credentials	Login is rejected	Pass
REG-003	Multiple invalid attempts	Attempts are tracked correctly	Pass
REG-004	Progressive lockout	Correct lockout is applied	Pass
REG-005	Login during lockout	Login remains blocked	Pass
REG-006	Lockout message	Message matches lockout duration	Pass
REG-007	Session timeout	Correct authentication state displayed	Pass

Confidentiality: This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.

ID	Regression Test	Expected Result	Result
REG-008	Account recovery	User can recover according to requirements	Pass
REG-009	Existing login workflow	Existing functionality remains unaffected	Pass

Regression result: PASS — the updated logout implementation and defect fixes did not introduce regressions across the tested authentication workflows.

QA Validation Flow

Retest confirms the defect was fixed. Regression confirms the fix did not break related functionality.

- 1. Verify the fix — confirm the reported defect is resolved.
- 2. Test boundaries — validate each progressive logout threshold.
- 3. Validate related states — check session timeout, account state, and login behavior.
- 4. Regression test — verify existing authentication workflows remain unaffected.
- 5. Release validation — confirm critical scenarios pass before release.

Scenarios Covered

- Successful login
- Invalid credentials
- Login attempt tracking
- Progressive logout
- Logout duration
- Logout messaging
- Session timeout
- Account recovery
- Authentication UI
- Existing login workflows

QA Traceability

A simple requirements-to-retest matrix showing how QA artifacts connect.

Requirement	Test Case	Defect	Retest
Progressive lockout	TC-004 to TC-010	BUG-001	RT-001 to RT-007
Correct lockout message	TC-004 to TC-010	BUG-001	RT-001 to RT-007
Session timeout behavior	TC-012	BUG-002	Regression
Existing login unaffected	TC-001 to TC-003	Regression scope	REG-001 to REG-009

Resolution

Simplifying the Lockout Implementation

The team simplified two lockout configurations into one mechanism while keeping the updated progressive lockout behavior.

- Simplified configuration
- Progressive lockout logic
- Correct lockout duration
- Correct user message
- Expected session behavior

QA revalidated the complete authentication flow after the implementation was simplified.

Regression Scope

Key areas validated after fixes. A fix was not considered complete until the affected functionality and related workflows were regression tested.

Test Execution Summary

- 12 representative test scenarios documented
- 7 retest scenarios documented
- 9 regression scenarios documented
- 2 key defects identified
- 100% of the showcased retest scenarios passed
- 100% of the showcased regression scenarios passed

***Confidentiality:** This case study is based on professional QA experience and has been anonymized for portfolio purposes. All company names, application names, screenshots, credentials, source code, proprietary workflows, internal data, and implementation-specific details have been excluded to protect confidential information.*

QA Recommendation

Based on the completed retesting and regression validation, the updated authentication behavior met the documented acceptance criteria across the tested scenarios. The lockout messaging and related authentication workflows were validated after the implementation changes.

Release Recommendation: PASS — ready for release validation based on the tested scope and documented results.

Outcome & Key Takeaways

QA identified inconsistencies in lockout messaging and session-state behavior during validation of the progressive lockout enhancement. After the implementation was simplified, the affected scenarios were retested and related authentication workflows were regression tested successfully.

Problem identified → Fix implemented → Retest passed → Regression passed → Release confidence.

QA Insight

The enhancement demonstrated that authentication changes should be tested as a complete state-based workflow rather than as isolated login scenarios.

- State consistency matters — attempt count, account state, lockout duration, message, and session state must remain aligned.
- Boundary scenarios expose hidden defects — the transition points between lockout levels were especially important.
- Fix validation requires regression — a successful retest confirms the defect was fixed, while regression confirms related functionality remains stable.

Collaboration

Role	Contribution
QA Analyst	Test execution, defect reporting, retesting, regression
Developer	Investigation and implementation of fixes
Scrum Master	Sprint coordination and tracking
Release Manager / Technical Project Manager	Release coordination and project validation

Skills and Tools

QA Skills

- Manual testing
- Functional testing
- Regression testing
- Negative testing
- Boundary testing
- Security testing
- Test case design
- Defect management
- Retesting
- Release validation

Tools

Azure DevOps: Test case and defect management

Swagger: API documentation and analysis

Postman: API testing and validation

Cursor AI: AI-assisted analysis and productivity